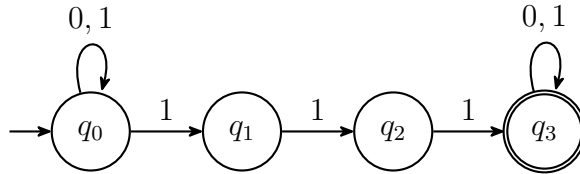


# Bài tập tổng hợp

Môn: Lý thuyết tính toán & Trình biên dịch

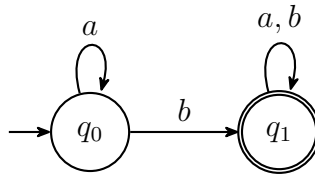
## Bài tập Tổng hợp

1. Cho NFA như hình bên dưới, hãy tìm DFA tương đương



2. Dựng NFA tương đương cho biểu thức chính quy  $R = a^*b \cup a$  sử dụng Thompson.

3. Chuyển DFA thành biểu thức



4. Cho văn phạm  $E \rightarrow *E \mid E[E] \mid id$ . Chỉ ra tính mơ hồ của chuỗi  $*id[id]$  và khử mơ hồ văn phạm đã cho.

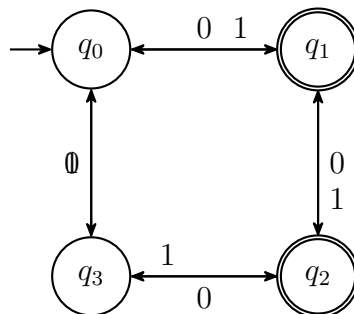
5. Cho văn phạm  $S \rightarrow aSb \mid bSa \mid SS \mid \epsilon$

- CMR văn phạm đã cho mơ hồ.
- Vẽ cây trái nhất để dựng chuỗi  $aabababb$ .
- Khử mơ hồ của văn phạm đã cho.

6. Chứng minh ngôn ngữ  $L_1 = \{a^n b^n \mid n \geq 0\}$  không chính quy.

7. Chứng minh ngôn ngữ  $L_2 = \{w \in \{0,1\}^* \mid |w| = 2^k, k \geq 0\}$  không chính quy.

8. Tối thiểu hoá DFA sau



- 4.2. Cho văn phạm  $P \rightarrow \epsilon \mid PP \mid (P)$ . Vẽ cây phân tích  $()()$  và  $(( ))$ .

4.3. Khử mơ hồ cho văn phạm  $E \rightarrow E + E \mid E * E \mid int$  trên chuỗi  $2+3*4$ .

9. Thiết kế máy Turing chấp nhận chuỗi trên  $\{0,1\}^*$  có ít nhất một ký tự '1'.

10. Thiết kế máy Turing kiểm tra chuỗi nhị phân có số lượng ký tự '0' và '1' bằng nhau.

11. Cho văn phạm  $A \rightarrow Ab \mid c$ . Khử đệ quy trái, tính FIRST, FOLLOW và lập bảng phân tích cú pháp LL(1).

12. Lập bảng phân tích cú pháp:

a) 1.  $E \rightarrow TY$

2.  $T \rightarrow int$

3.  $T \rightarrow (E)$

4.  $Y \rightarrow +E$

5.  $Y \rightarrow \epsilon$

b)  $S \rightarrow uBDz$

$B \rightarrow Bv \mid w$

$D \rightarrow EF$

$E \rightarrow y \mid \epsilon$

$F \rightarrow z \mid \epsilon$

13. Tìm kiếm Handle

$S \rightarrow E$

$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow int$

$T \rightarrow (E)$

14. Xây dựng automata LR(1)

$S \rightarrow E$

$E \rightarrow T$

$E \rightarrow E + T$

$T \rightarrow int$

$T \rightarrow (E)$

15. Cho văn phạm  $S \rightarrow aSb \mid bSa \mid SS \mid \epsilon$

a. Vẽ cây trái nhất chuỗi  $aabababb$

b. CMR văn phạm đã cho mơ hồ

c. Khử mơ hồ theo phương pháp phân tầng

16. Cho văn phạm:

$S \rightarrow SP \mid P$

$P \rightarrow (S) \mid ()$

a. Xây dựng LR(1)

b. Cho biết trạng thái accept, reduce, có đùng độ không?

13. Tìm kiếm Handle

$$\begin{aligned} S &\rightarrow E \\ E &\rightarrow T \\ E &\rightarrow T + E \\ T &\rightarrow int \\ T &\rightarrow (E) \end{aligned}$$

14. Xây dựng automata LR(1)

$$\begin{aligned} S &\rightarrow E \\ E &\rightarrow T \\ E &\rightarrow E + T \\ T &\rightarrow int \\ T &\rightarrow (E) \end{aligned}$$

15. Cho văn phạm  $S \rightarrow aSb \mid bSa \mid SS \mid \epsilon$

- Vẽ cây trái nhất chuỗi *aabababb*
- CMR văn phạm đã cho mơ hồ
- Khử mơ hồ theo phương pháp phân tầng

16. Cho văn phạm:

$$\begin{aligned} S &\rightarrow SP \mid P \\ P &\rightarrow (S) \mid () \end{aligned}$$

- Xây dựng LR(1)
- Cho biết trạng thái accept, reduce, có dụng độ không?

## PHẦN 1: AUTOMATA VÀ NGÔN NGỮ CHÍNH QUY

### Bài 1: Chuyển NFA sang DFA (Subset Construction)

Giải chi tiết từng bước:

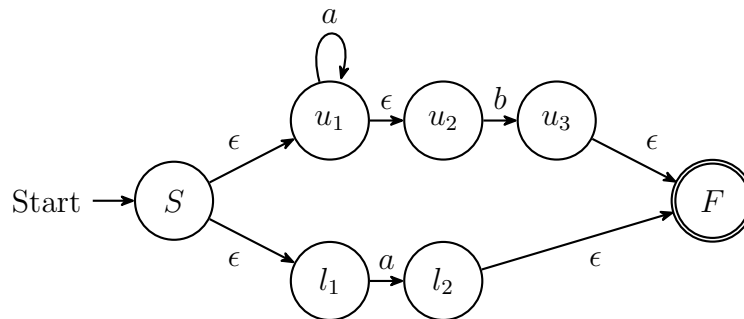
| Trạng thái DFA (Tập hợp)      | Nhập 0             | Nhập 1                       | Chấp nhận? |
|-------------------------------|--------------------|------------------------------|------------|
| $\rightarrow A = \{q_0\}$     | $\{q_0\} = A$      | $\{q_0, q_1\} = B$           | Không      |
| $B = \{q_0, q_1\}$            | $\{q_0\} = A$      | $\{q_0, q_1, q_2\} = C$      | Không      |
| $C = \{q_0, q_1, q_2\}$       | $\{q_0\} = A$      | $\{q_0, q_1, q_2, q_3\} = D$ | Không      |
| $*D = \{q_0, q_1, q_2, q_3\}$ | $\{q_0, q_3\} = E$ | $\{q_0, q_1, q_2, q_3\} = D$ | <b>Có</b>  |
| $*E = \{q_0, q_3\}$           | $\{q_0, q_3\} = E$ | $\{q_0, q_1, q_3\} = F$      | <b>Có</b>  |
| $*F = \{q_0, q_1, q_3\}$      | $\{q_0, q_3\} = E$ | $\{q_0, q_1, q_2, q_3\} = D$ | <b>Có</b>  |

Giải thích tại sao làm vậy (Bản chất):

- **Tại sao trạng thái DFA lại là một tập hợp?** Vì máy NFA (không đơn định) khi đọc 1 ký tự có thể phân thân ra nhiều trạng thái cùng lúc. Ví dụ: Từ  $q_0$  đọc 1, nó vừa đứng yên tại  $q_0$  (nhờ vòng lặp), vừa nhảy sang  $q_1$ . Do đó ta gom chúng lại thành tập  $\{q_0, q_1\}$  để tạo thành 1 trạng thái  $B$  duy nhất cho DFA.
- **Cách tính cột Nhập 0/1:** Lấy từng phần tử trong tập hợp, xem nó đi đâu rồi hợp (Union) lại. Ví dụ: Tính  $B = \{q_0, q_1\}$  khi nhập 1:  $q_0$  đi tới  $\{q_0, q_1\}$ ,  $q_1$  đi tới  $q_2$ . Gộp lại ta được  $C = \{q_0, q_1, q_2\}$ .
- **Tại sao lại Accept?** Trạng thái kết thúc của NFA ban đầu là  $q_3$ . Do đó, cứ tập hợp nào chứa  $q_3$  (như  $D, E, F$ ) thì nó mặc nhiên trở thành trạng thái Chấp nhận (Accept) của DFA.

## Bài 2: Dựng NFA cho $R = a^*b \cup a$ bằng Thompson

Giải chi tiết từng bước:



**Giải thích tại sao làm vậy (Bản chất):**

- **Toán tử  $\cup$  (Hợp):** Biểu thức là  $X \cup Y$ . Theo luật Thompson, ta tạo 1 trạng thái bắt đầu  $S$  và 1 trạng thái kết thúc  $F$ . Từ  $S$  rẽ làm 2 nhánh bằng bước nhảy rỗng ( $\epsilon$ ), 1 nhánh đi vẽ cho  $X$  (tức  $a^*b$ ), 1 nhánh vẽ cho  $Y$  (tức  $a$ ), sau đó hội tụ lại về  $F$  bằng  $\epsilon$ .
- **Nhánh  $a^*b$ :** Gồm phép lặp  $a^*$  ghép nối (concatenation) với  $b$ . Do đó  $u_1$  có vòng lặp  $a$ , sau đó dùng  $\epsilon$  nối sang  $u_2$  để đọc tiếp  $b$ .

## Bài 3: Chuyển DFA thành biểu thức chính quy (RE)

Giải chi tiết từng bước:

Biểu thức chính quy là:  $R = a^*b(a \cup b)^*$

**Giải thích tại sao làm vậy (Bản chất):**

- **Giai đoạn 1 (Tại  $q_0$ ):** Bắt đầu tại  $q_0$ , máy có một vòng lặp nhân  $a$ . Có nghĩa là nó có thể nhận 0, 1 hoặc nhiều chữ  $a$ . Ta biểu diễn là  $a^*$ .
- **Giai đoạn 2 (Chuyển tiếp):** Để đi đến trạng thái kết thúc  $q_1$ , bắt buộc phải đi qua 1 cung có nhân  $b$ . Ta nối thêm chữ  $b$  vào biểu thức  $\rightarrow a^*b$ .
- **Giai đoạn 3 (Tại  $q_1$ ):** Đã đến đích, nhưng tại đây có vòng lặp nhân  $a, b$ . Có nghĩa là chuỗi có thể tiếp tục bằng bất kỳ sự kết hợp nào của  $a$  và  $b$  mà vẫn được Accept. Biểu diễn là  $(a \cup b)^*$ . Ghép cả 3 lại ta có kết quả.

## Bài 6 & 7: Bổ đề bơm chứng minh không chính quy

Giải chi tiết và Giải thích tại sao:

**Bài 6** ( $L_1 = \{a^n b^n \mid n \geq 0\}$ ):

- **Bước 1:** Giả sử  $L_1$  chính quy  $\implies$  tồn tại hằng số bơm  $p \geq 1$ .
- **Bước 2** (Tại sao chọn  $s = a^p b^p$ ?): Cố tình chọn chuỗi  $s$  có độ dài  $2p \geq p$  để ép nó phải tuân thủ Bổ đề bơm, giúp ta dễ tạo ra mâu thuẫn.
- **Bước 3** (Tách  $s = xyz$ ): Điều kiện bổ đề bắt buộc  $|xy| \leq p$ . Vì  $p$  ký tự đầu tiên của chuỗi  $s$  toàn là chữ  $a$ , nên đoạn  $y$  bắt buộc chỉ chứa toàn chữ  $a$  (giả sử  $y = a^m, m \geq 1$ ).
- **Bước 4** (Bơm): Bơm lên với  $k = 2 \implies s' = xy^2z$ . Lúc này số lượng chữ  $a$  bị nhân lên thành  $p + m$ , nhưng số chữ  $b$  ở đoạn  $z$  vẫn giữ nguyên là  $p$ . Do  $p + m \neq p \implies s' \notin L_1$  (Mâu thuẫn).

**Bài 7** ( $L_2 = \{w \in \{0, 1\}^* \mid |w| = 2^k\}$ ):

- **Bước 1 & 2:** Giả sử chính quy  $\implies$  có hằng số  $p$ . Chọn chuỗi  $s = 0^{2^p}$ . (Tại sao? Để độ dài  $|s| = 2^p \geq p$ , đủ dài để bơm).
- **Bước 3** (Tách chuỗi): Tách  $s = xyz$ . Đặt độ dài của phần bị bơm là  $|y| = m$ . Theo điều kiện bổ đề,  $1 \leq m \leq p$ .
- **Bước 4** (Bơm & Đánh giá): Bơm  $k = 2 \implies s' = xy^2z$ . Độ dài chuỗi mới là  $|s'| = 2^p + m$ .
- **Giải thích Mâu thuẫn:** Vì  $1 \leq m \leq p$ , ta có khoảng giá trị của  $|s'|$  là:  $2^p < |s'| \leq 2^p + p < 2^p + 2^p = 2^{p+1}$ . Tức là độ dài chuỗi mới nằm kẹt chặt giữa 2 lũy thừa của 2 liên tiếp. Do đó độ dài  $|s'|$  không thể là một lũy thừa của 2  $\implies s' \notin L_2$  (Mâu thuẫn).

## Bài 8: Tối thiểu hóa DFA

Giải chi tiết từng bước:

1. **Phân hoạch  $P_0$ :** Chia thành tập Chấp nhận ( $F$ ) và Không chấp nhận ( $NF$ ).  $P_0 = \{\{q_0, q_3\}, \{q_1, q_2\}\}$ .
2. **Phân hoạch  $P_1$  (Xét nhóm  $\{q_0, q_3\}$ ):** Dưới tác động của ký tự 0:  $q_0 \xrightarrow{0} q_1$  (thuộc nhóm  $F$ ).  $q_3 \xrightarrow{0} q_0$  (thuộc nhóm  $NF$ ).  $\implies$  Tách ra thành  $\{q_0\}, \{q_3\}$ .
3. **Phân hoạch  $P_1$  (Xét nhóm  $\{q_1, q_2\}$ ):** Dưới tác động của ký tự 0:  $q_1 \xrightarrow{0} q_2$  (thuộc nhóm  $F$ ).  $q_2 \xrightarrow{0} q_3$  (thuộc nhóm  $NF$ ).  $\implies$  Tách ra thành  $\{q_1\}, \{q_2\}$ .
4. **Kết luận:**  $P_1 = \{\{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}\}$ . Do mỗi nhóm chỉ còn 1 phần tử nên DFA đã cho **đã là DFA tối thiểu**.

**Giải thích tại sao làm vậy (Bản chất):**

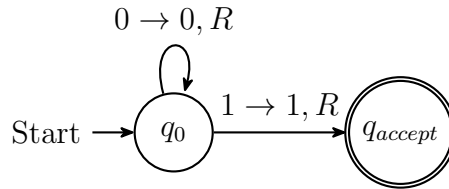
- **Tư tưởng tối thiểu hóa:** Hai trạng thái có thể bị gộp lại làm 1 nếu chúng "hành xử giống hệt nhau". Nghĩa là dù đọc ký tự nào, chúng cũng dắt tay nhau cùng đi về nhóm kết thúc, hoặc cùng đi về nhóm không kết thúc.
- **Tại sao phải tách?** Ở bước 2, tại sao tách  $q_0$  và  $q_3$ ? Vì nếu nhập số 0, anh  $q_0$  thì đưa máy đến trạng thái Accept ( $q_1$ ), còn anh  $q_3$  lại đưa máy về trạng thái từ chối ( $q_0$ ). Hành vi trái ngược nhau hoàn toàn nên không thể ở chung 1 nhóm, buộc phải tách riêng.

## PHẦN 2: MÁY TURING (TURING MACHINE)

### Bài 9: Thiết kế máy Turing chấp nhận chuỗi chứa ít nhất một chữ '1'

Giải chi tiết từng bước:

Ta thiết kế máy Turing  $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$  với sơ đồ chuyển trạng thái như sau:



Hàm chuyển  $\delta$ :

- $\delta(q_0, 0) = (q_0, 0, R)$
- $\delta(q_0, 1) = (q_{accept}, 1, R)$

**Giải thích tại sao làm vậy (Bản chất):**

- **Mục tiêu:** Chỉ cần tìm thấy **ít nhất một** số '1' là lập tức chấp nhận chuỗi.
- **Cách hoạt động:** Đầu đọc (Head) bắt đầu ở ngoài cùng bên trái ( $q_0$ ). Nó sẽ liên tục dịch sang phải ( $R$ ). Nếu nó đọc được số '0', nó kẻ nó (viết đè lại chính số '0') và đi tiếp (vòng lặp tại  $q_0$ ). Ngay giây phút nó đọc được số '1', nó lập tức chuyển sang trạng thái  $q_{accept}$  và dừng máy. Chuỗi được chấp nhận!

### Bài 10: Thiết kế máy Turing kiểm tra số lượng '0' và '1' bằng nhau

Giải chi tiết từng bước (Thuật toán đánh dấu chéo):

1. **Bước 1 (Tìm 0):** Ở trạng thái  $q_0$ , đầu đọc quét từ trái sang phải. Bỏ qua các ký tự 'X', 'Y'. Nếu thấy '0' đầu tiên, ghi đè nó thành 'X' (đã đánh dấu) và chuyển sang trạng thái  $q_1$  (đi tìm 1).
2. **Bước 2 (Tìm 1 tương ứng):** Ở trạng thái  $q_1$ , tiếp tục đi sang phải, bỏ qua các số '0' và chữ 'Y'. Nếu thấy '1' đầu tiên, ghi đè nó thành 'Y' và chuyển sang trạng thái  $q_2$  (quay về).
3. **Bước 3 (Quay về đầu):** Ở  $q_2$ , đầu đọc lùi liên tục về bên trái ( $L$ ) cho đến khi đụng phải ký tự khoảng trắng ( $\sqcup$ ) hoặc chữ 'X' ngoài cùng. Sau đó tiến sang phải 1 bước, trở lại trạng thái  $q_0$  và lặp lại Bước 1.
4. **Bước 4 (Kết thúc):** Nếu ở  $q_0$  mà máy chỉ còn nhìn thấy toàn 'Y' và khoảng trắng  $\sqcup$  (tức là không còn số '0' hay '1' nào bị thừa), máy chuyển sang  $q_{accept}$ .

**Giải thích tại sao làm vậy (Bản chất):**

- **Tại sao không đếm?** Máy Turing không có biến đếm *count* như các ngôn ngữ lập trình thông thường. Bộ nhớ của nó chính là băng giấy.
- **Tại sao lại đánh dấu X và Y?** Để kiểm tra số lượng bằng nhau, ta dùng thuật toán "Bắt cặp". Cứ xóa (đánh dấu) một chữ '0' thì phải chạy đi tìm một chữ '1' để xóa (đánh dấu) theo. Chữ 'X' và 'Y' giúp máy nhớ rằng vị trí đó đã được bắt cặp rồi, lần duyệt sau sẽ bỏ qua. Nếu cuối cùng mọi thứ bắt cặp hoàn hảo, băng giấy chỉ còn lại X và Y  $\implies$  số lượng bằng nhau.

# PHẦN 3: VĂN PHẠM VÀ KHỬ MƠ HỒ

## Bài 4: Khử mơ hồ cho toán tử con trỏ và mảng

**Đề bài:** Cho văn phạm  $E \rightarrow *E \mid E[E] \mid id$ . Chỉ ra tính mơ hồ của chuỗi  $*id[id]$  và khử mơ hồ.

**Giải chi tiết từng bước:**

- **Chỉ ra tính mơ hồ:** Chuỗi  $*id[id]$  có thể được sinh ra từ 2 cây dẫn xuất trái nhất khác nhau:

– Dẫn xuất 1 (Lấy mảng trước):  $E \Rightarrow *E \Rightarrow *E[E] \Rightarrow *id[E] \Rightarrow *id[id]$

– Dẫn xuất 2 (Lấy con trỏ trước):  $E \Rightarrow E[E] \Rightarrow *E[E] \Rightarrow *id[E] \Rightarrow *id[id]$

- **Khử mơ hồ (Ưu tiên mảng  $[]$  cao hơn con trỏ  $*$ ):**

Ta phân tầng văn phạm thành các mức độ ưu tiên:

$$E \rightarrow *E \mid T$$

$$T \rightarrow T[E] \mid F$$

$$F \rightarrow id \mid (E)$$

**Giải thích tại sao làm vậy (Bản chất):**

- **Tại sao lại mơ hồ?** Vì văn phạm gốc gộp chung cả toán tử  $*$  và  $[]$  vào cùng một biến  $E$ . Máy tính không biết chuỗi này nên được hiểu là "Lấy giá trị tại địa chỉ của  $id[id]$ " (tức là  $*(id[id])$ ) hay "Phần tử mảng của một con trỏ" (tức là  $(*id)[id]$ ).
- **Tại sao lại phân tầng (Stratification)?** Trong thiết kế trình biên dịch, toán tử nào có độ ưu tiên cao hơn thì phải nằm ở **sâu hơn** trong cây phân tích (tức là sinh ra sau cùng). Bằng cách cho  $T \rightarrow T[E]$ , ta ép phép lấy mảng  $[]$  được tính toán trước. Ký hiệu  $*$  nằm ở biến  $E$  (gần gốc cây hơn) nên sẽ được tính sau.

## Bài 4.3: Khử mơ hồ biểu thức toán học

**Đề bài:** Khử mơ hồ cho văn phạm  $E \rightarrow E + E \mid E * E \mid int$  trên chuỗi  $2+3*4$ .

**Giải chi tiết từng bước:**

Để khử mơ hồ, ta thiết lập độ ưu tiên: Dấu ngoặc  $()$  > Phép nhân  $*$  > Phép cộng  $+$ . Ta viết lại văn phạm:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow int \mid (E)$$

**Giải thích tại sao làm vậy (Bản chất):**

- Nếu dùng văn phạm gốc, chuỗi  $2+3*4$  có thể bị máy tính hiểu lầm là  $(2+3)*4 = 20$  (nếu nó tính phép cộng trước) thay vì  $2+(3*4) = 14$ .
- Bằng cách tạo ra các biến trung gian  $T$  (Term - Hạng tử) và  $F$  (Factor - Nhân tử), ta ép máy tính phải đi sâu xuống đáy cây để thực hiện phép nhân ( $T \rightarrow T * F$ ) trước khi ngoi lên trên để thực hiện phép cộng ( $E \rightarrow E + T$ ). Đồng thời, đệ quy trái ( $E \rightarrow E + T$ ) giúp đảm bảo tính kết hợp từ trái sang phải (Left-associative).

## Bài 5 (và 15): Phân tích văn phạm $S \rightarrow aSb \mid bSa \mid SS \mid \epsilon$

Giải chi tiết từng bước:

- **a. CMR văn phạm mơ hồ (với chuỗi abab):** Ta tìm được 2 dẫn xuất trái nhất:

– Cây 1:  $S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abab$

– Cây 2:  $S \Rightarrow aSb \Rightarrow abSab \Rightarrow abab$

- **b. Dẫn xuất trái nhất cho chuỗi aabababb:**

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabSabb \Rightarrow aabaSbabb \Rightarrow aabababb$

- **c. Khử mơ hồ:**

Ngôn ngữ này mô tả các chuỗi có số lượng chữ 'a' và 'b' bằng nhau. Văn phạm không mơ hồ kinh điển thay thế cho nó là:

$$S \rightarrow \epsilon \mid aSbS \mid bSaS$$

**Giải thích tại sao làm vậy (Bản chất):**

- **Nguyên nhân mơ hồ:** Sự xuất hiện của luật sinh  $S \rightarrow SS$ . Nó cho phép một chuỗi bị bẻ gãy thành 2 nửa một cách tùy tiện. Ví dụ chuỗi abab có thể coi là 1 khối thống nhất sinh từ  $aSb$ , hoặc bị bẻ thành 2 khối độc lập ghép lại sinh từ  $SS$  (khối ab và khối ab).
- **Tại sao văn phạm mới lại hết mơ hồ?** Bằng cách ép chữ  $a$  hoặc  $b$  làm "mỏ neo" ở đầu ( $aSbS$  hoặc  $bSaS$ ), ta bắt buộc mỗi khi sinh ra một chữ  $a$ , nó phải tìm ngay một chữ  $b$  để bắt cặp (phần  $S$  ở giữa), và sau đó mới sinh tiếp đoạn chuỗi còn lại (phần  $S$  ở cuối). Điều này triệt tiêu hoàn toàn sự rẽ nhánh vô tổ chức của  $S \rightarrow SS$ .

## PHẦN 4: PHÂN TÍCH CÚ PHÁP LL(1)

### Bài 11 & 12: Khử đệ quy trái và Lập bảng LL(1)

Bài 11: Cho văn phạm  $A \rightarrow Ab \mid c$

1. Giải chi tiết từng bước:

- **Khử đệ quy trái:** Áp dụng công thức, ta có văn phạm mới:

$$A \rightarrow cA'$$

$$A' \rightarrow bA' \mid \epsilon$$

- **Tính FIRST & FOLLOW:**

–  $FIRST(A) = \{c\}$  (vì từ A đi ra luôn bắt đầu bằng c).

–  $FIRST(A') = \{b, \epsilon\}$ .

–  $FOLLOW(A) = \{\$ \}$  (vì A là biến bắt đầu).

–  $FOLLOW(A') = FOLLOW(A) = \{\$ \}$  (vì A' đứng cuối luật sinh của A).

**Bài 12a: Cho  $E \rightarrow TY$ ,  $T \rightarrow int \mid (E)$ ,  $Y \rightarrow +E \mid \epsilon$**

1. Giải chi tiết từng bước:

- **Tính FIRST:**  $FIRST(T) = \{int, (\}$ ,  $FIRST(Y) = \{+, \epsilon\}$ . Vì  $E \rightarrow TY$ , E kế thừa FIRST của T  $\implies FIRST(E) = \{int, (\}$ .



• Tính FOLLOW:

- $FOLLOW(E) = \{\$, )\}$ . (Có \$ vì là biến bắt đầu; có ) vì  $T \rightarrow (E)$ ).
- $FOLLOW(Y) = FOLLOW(E) = \{\$, )\}$  (Vì Y đứng cuối luật  $E \rightarrow TY$ ).
- $FOLLOW(T) = FIRST(Y \setminus \{\epsilon\}) \cup FOLLOW(E) = \{+, \$, )\}$ .

2. Bảng phân tích cú pháp LL(1):

|   | int                 | +                  | (                   | )                        | \$                       |
|---|---------------------|--------------------|---------------------|--------------------------|--------------------------|
| E | $E \rightarrow TY$  |                    | $E \rightarrow TY$  |                          |                          |
| T | $T \rightarrow int$ |                    | $T \rightarrow (E)$ |                          |                          |
| Y |                     | $Y \rightarrow +E$ |                     | $Y \rightarrow \epsilon$ | $Y \rightarrow \epsilon$ |

Giải thích tại sao làm vậy (Bản chất):

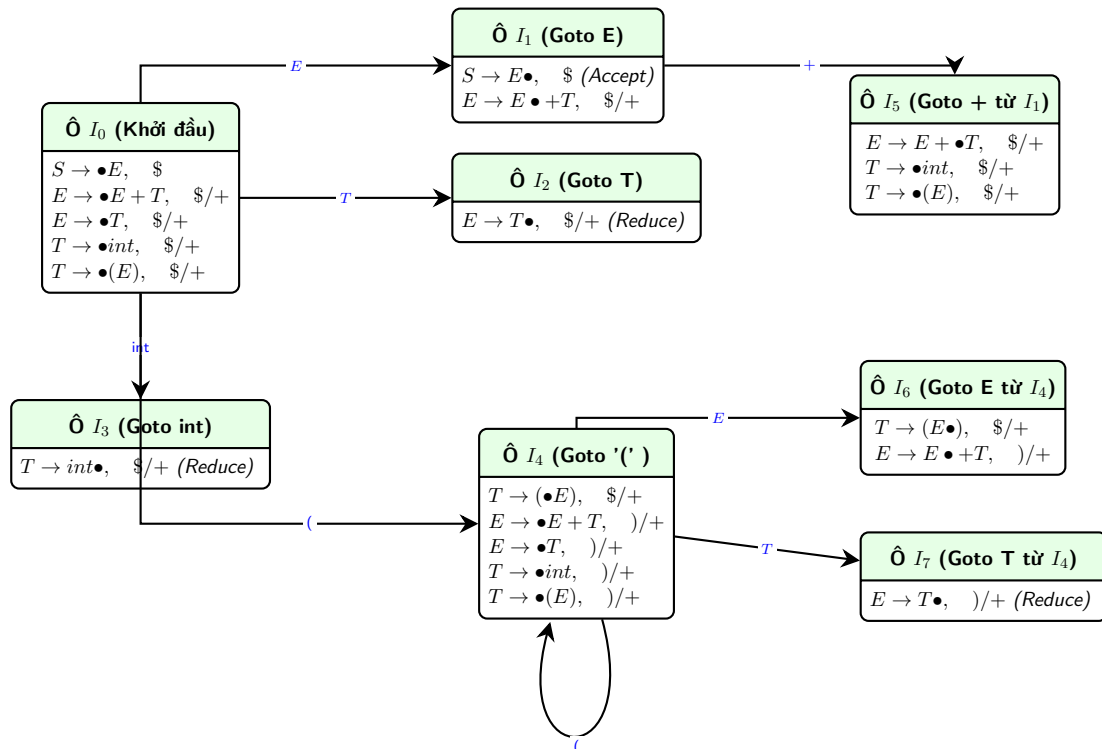
- **Tại sao phải khử đệ quy trái?** Bộ phân tích LL(1) đọc từ trên xuống, từ trái sang phải. Nếu có luật  $A \rightarrow Ab$ , nó sẽ liên tục gọi A, để ra A, rồi lại để ra A sinh ra vòng lặp vô hạn khiến máy tính bị treo.
- **Tại sao phải tính FIRST và FOLLOW?** Để lập bảng. Nguyên tắc lập bảng là: Khi đứng ở biến X, nếu nhìn thấy ký tự a thuộc  $FIRST(X)$ , ta sẽ dùng luật sinh tạo ra a. Nếu luật sinh đó tạo ra  $\epsilon$  (rỗng), ta sẽ dùng nó khi nhìn thấy các ký tự nằm trong tập  $FOLLOW(X)$  (tức là những ký tự được phép đứng ngay sau X).

## PHẦN 5: PHÂN TÍCH CÚ PHÁP LR (BOTTOM-UP)

### Bài 14: Xây dựng Automaton LR(1) cho biểu thức toán học

Luật sinh:  $S \rightarrow E$ ,  $E \rightarrow E + T \mid T$ ,  $T \rightarrow int \mid (E)$ .

1. Giải chi tiết từng bước (Vẽ Automata đầy đủ các nhánh cốt lõi):



*\*(Ghi chú: Đồ thị đã thể hiện đầy đủ mọi hành vi Shift, Reduce, Goto và cách tính Lookahead. Các nhánh đọc 'int' từ  $I_4, I_5$  sẽ sinh ra trạng thái Reduce tương tự  $I_3$  nhưng khác Lookahead).\**

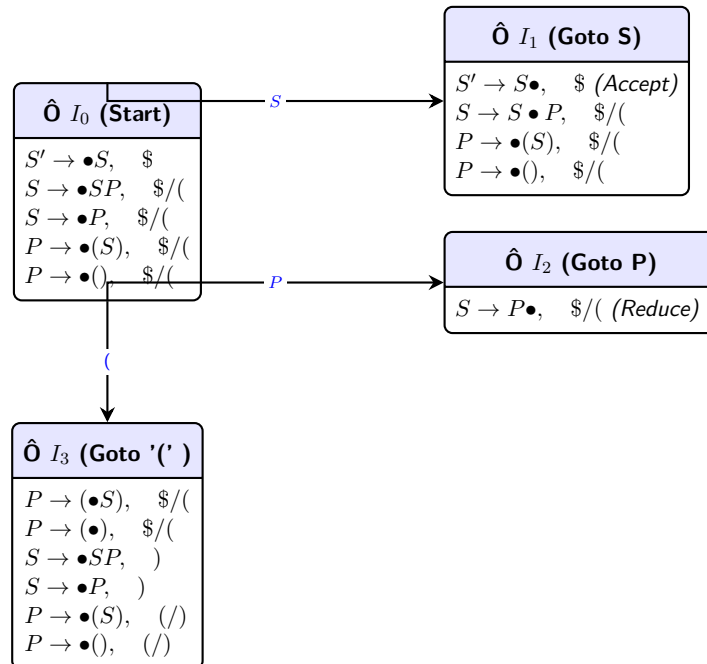
**Giải thích tại sao làm vậy (Bản chất của LR(1)):**

- **Cấu trúc một mục LR(1):** Gồm phần luật sinh và phần Lookahead:  $[A \rightarrow \alpha \bullet B\beta, a]$ . Lookahead  $a$  mang ý nghĩa "Những ký tự nào được phép xuất hiện ngay sau luật sinh này để quyết định thu gọn (Reduce)".
- **Luật tính Lookahead (Tại sao  $I_4$  lại ra  $)/+?$ ):** Khi ở  $I_4$ , ta có  $T \rightarrow (\bullet E), \$ / +$ . Ở đây  $\beta$  là dấu ')',  $a$  là  $\$ / +$ . Theo công thức, Lookahead của  $E$  sẽ là  $FIRST()\$ / +) = )$ . Tiếp tục từ  $E \rightarrow \bullet E + T, )$ , ta lại có  $\beta$  là  $+T$ ,  $a$  là  $)$ . Lookahead của  $T$  sẽ là  $FIRST(+T)) = +$ . Gộp lại ta có tập Lookahead mới là  $)/+$ .

## Bài 16: LR(1) cho $S \rightarrow SP \mid P, \quad P \rightarrow (S) \mid ()$

### 1. Giải chi tiết từng bước:

- Tính  $FIRST(P) = \{ ( \}$ .
- Xây dựng các trạng thái cốt lõi:



**Giải thích tại sao làm vậy (Bản chất Đánh giá xung đột):**

- **Bao đóng tại  $I_3$ :** Ở  $I_3$ , luật mới là  $P \rightarrow (\bullet S), \$ / ($ . Theo công thức, Lookahead cho  $S$  là  $FIRST()\$ / ( = )$ . Do đó  $S$  mang Lookahead là  $)$ ; tiếp tục sinh ra  $P$  mang Lookahead là  $(/$ . Sự thay đổi Lookahead này là tinh hoa của LR(1), giúp nó phân biệt được ngữ cảnh.
- **Đánh giá Conflict (Đụng độ):** Văn phạm này **hoàn toàn không có đụng độ (No Conflict)**. Nhìn vào  $I_1$ , nó vừa có trạng thái Accept ( $S' \rightarrow S \bullet$ ), vừa có lệnh Shift đi tiếp (đọc  $P$ ). Tuy nhiên, Accept chỉ xảy ra nếu ký tự tiếp theo đọc được là dấu kết thúc  $\$$ , còn Shift chỉ xảy ra nếu đọc được dấu ngoặc mở ' $($ '. Ký hiệu nhìn trước (Lookahead) khác biệt nhau hoàn toàn nên máy luôn biết phải làm gì, không bị nhầm lẫn.

## "Công thức vàng" xác định Lookahead trong LR(1)

Giả sử trong một trạng thái, bạn đang xét một dòng có dạng:

$$[A \rightarrow \alpha \bullet B\beta, \quad a]$$

(Trong đó: dấu  $\bullet$  đứng trước biến  $B$ ,  $\beta$  là những gì đứng sau  $B$ , và  $a$  là Lookahead hiện tại của dòng này)

Khi khai triển (tính Bao đóng) các luật sinh của biến  $B$ , Lookahead mới cho các luật của  $B$  sẽ được tính bằng công thức:

$$\text{Lookahead}_{\text{mi}} = \text{FIRST}(\beta a)$$

**Dịch ra ngôn ngữ "Thực chiến" dễ hiểu:**

Muốn biết Lookahead của  $B$  là gì, hãy nhìn **kẻ đứng ngay sau lưng  $B$**  (tức là  $\beta$ ):

- **Trường hợp 1 - Sau lưng  $B$  KHÔNG CÓ GÌ ( $\beta$  rỗng):**  
Luật sinh của  $B$  sẽ **kế thừa y nguyên** dấu  $a$  từ dòng trên rớt xuống.
- **Trường hợp 2 - Sau lưng  $B$  CÓ CHỮ/DẤU ( $\beta \neq \epsilon$ ):**  
Luật sinh của  $B$  sẽ nhận dấu là tập **FIRST**( $\beta$ ). (Nếu **FIRST**( $\beta$ ) có thể sinh ra rỗng thì nhận thêm cả dấu  $a$ ).